

The Fourth-Order Bratu Problem

Critical Parameters of the Fourth-Order Bratu Problem, Computed and Verified

Luminary Quantum Institute, LLC (dba Mitra Institute of Education)

SIRPY · Problem 15

See problem 14 - The Second-Order Bratu Problem

An ongoing series. We are building a public library of problems SIRPY has solved — each entry is one problem, the exact input we handed the solver, the exact output it returned, and an honest read of what happened. No slides. No hand-waving. Numbers you can check.

Abstract

The fourth-order Bratu (Gelfand) problem, $y^{(4)} = \lambda e^y$, models the deflection of a beam under a load that grows exponentially with the deflection itself. Like its classical second-order relative it possesses a fold: below a critical parameter λ_c two solutions exist, at λ_c exactly one, and above it none. We report λ_c for the two standard end conditions — Navier (simply supported) and clamped — computed with SIRPY, a solver that returns, alongside every answer, a measurement of how well that answer satisfies the equation and the boundary data. Each solution reported here satisfies its equation to a relative residual below 10^{-13} and its boundary data below 10^{-15} , by exact differentiation rather than by finite differences. The critical values are

- **Navier**, $y(0)=y(1)=y'(0)=y'(1)=0$: $\lambda_c \approx 34.6402$
- **Clamped**, $y(0)=y(1)=y'(0)=y'(1)=0$: $\lambda_c \approx 176.4714$

Each is obtained twice, by methods that share neither discretisation nor algorithm. A shooting scan brackets the fold between the last converging λ and the first failing one; a pseudo-arclength continuation then *locates* it as a limit point, traversing the turning point onto the upper branch. The two agree to 0.03 % (Navier) and 0.004 % (clamped). Both are stable under refinement, so they belong to the differential problem rather than to a discretisation. We also record a structural observation: across the second-order problem and both fourth-order

end conditions, the product of λ_c with the compliance of the corresponding operator is nearly constant (0.439, 0.451, 0.460), and the solution amplitude at the fold is nearly the same in all three ($\|y\|_\infty \approx 1.19$)¹

Finally, we draw attention to a sign convention that is easy to misread and which changes the problem completely: written as $y^{(4)} + \lambda e^y = 0$, by false analogy with the second-order Bratu equation, the problem has **no** critical parameter at all.

1. The problem and its history

The **second-order Bratu** problem

$$u'' + \lambda e^u = 0, \quad u(0) = u(1) = 0 \quad (1)$$

arises in models of thermal self-ignition, and is one of the most heavily used benchmarks in numerical analysis because it can be solved in closed form. It has two solutions for $\lambda < \lambda_c$, one at $\lambda = \lambda_c$, and none for $\lambda > \lambda_c$, with

$$\lambda_c = 3.513830719125 \dots \quad (2)$$

The **fourth-order** counterpart replaces the string by a beam:

$$y^{(4)} = \lambda e^y, \quad 0 < x < 1 \quad (3)$$

with either **Navier** (simply supported) end conditions

$$y(0) = y(1) = 0, \quad y''(0) = y''(1) = 0 \quad (4)$$

or **clamped** end conditions

¹For the second-order problem, the maximum occurs at $x = \frac{1}{2}$. At the fold,

$$\beta \tanh\left(\frac{\beta}{4}\right) = 4, \quad \beta^* \approx 4.7987,$$

so

$$\lambda_c = \frac{(\beta^*)^2}{2 \cosh^2(\beta^*/4)} \approx 3.5138, \quad |y|_\infty = y! \left(\frac{1}{2}\right) = 2 \ln \cosh\left(\frac{\beta^*}{4}\right) \approx \boxed{1.1868} \approx 1.19.$$

For the fourth-order problem, $|y|_\infty$ is computed numerically as $|y|_\infty = \max_{x \in [0, 1]} |y(x)|$, evaluated on the computed solution near the critical λ .

$$y(0) = y(1) = 0, \quad y'(0) = y'(1) = 0 \quad (5)$$

Physically, (3)–(4) is a beam resting on two supports under a load that increases exponentially with deflection; (3)–(5) is the same beam built into its supports at both ends. As in the second-order case, the exponential feeds on the deflection it produces, and beyond a critical load parameter no equilibrium exists.

1.1 A sign convention that reverses the problem

Equation (1) is customarily written with the nonlinearity on the left and a plus sign, because the positive operator in second order is $-d^2/dx^2$. In fourth order the positive operator is $+d^4/dx^4$, so the same physics is written as (3), with the fourth derivative alone on the left. This is the form used throughout the literature on the fourth-order Gelfand problem and on simply supported beam equations.

Transcribing the second-order sign convention directly into fourth order — writing $y^{(4)} + \lambda e^y = 0$ — silently produces a *different* problem, and one with no fold whatsoever. The argument is two lines. Put $u = y''$. Then $u'' = y^{(4)} = -\lambda e^y < 0$, so u is concave and vanishes at both ends, whence $u \geq 0$; therefore $y'' \geq 0$, so y is convex and vanishes at both ends, whence

$y \leq 0$ on $[0,1]$ for every $\lambda > 0$ and every solution. (6)

But $y \leq 0$ gives $e^y \leq 1$, so the load is bounded by λ and cannot amplify itself. The nonlinearity saturates instead of exploding, and solutions exist for **all** $\lambda > 0$. (In the elliptic setting this positivity is Boggio's principle: $\Delta^2 u \geq 0$ with clamped data forces $u \geq 0$.)

We verified this directly. With the sign of (6), SIRPY returns verified solutions at $\lambda = 75, 100, 200, 7\,500, 15\,000, 50\,000, 100\,000$ and $250\,000$, with y negative throughout in every case (e.g. $y(1/2) = -0.6085$ at $\lambda = 75$, $y(1/2) = -0.7363$ at $\lambda = 100$), and with $y'(0)$ growing smoothly like $\lambda^{1/4}$ and $y'''(0)$ like $\lambda^{3/4}$ — the signature of a boundary layer of width $\lambda^{-1/4}$, not of an approaching fold. Everything that follows therefore concerns equation (3) as written.

2. How the problems were entered

No reformulation, no first-order system, no initial guess, and no continuation strategy were supplied. The equation and the boundary conditions are typed as they appear in Section 1. Unknown boundary values are named rather than guessed.

Navier:

```

from sirpy import solve_bvp

r = solve_bvp(order=4, f_str=f"{lam}*exp(y)", x0=0, x1=1,
               left_bc={0: 0.0, 1: "gamma", 2: 0.0, 3: "eta"},
               right_bc={0: 0.0, 2: 0.0},
               partition=16)

```

Clamped:

```

r = solve_bvp(order=4, f_str=f"{lam}*exp(y)", x0=0, x1=1,
               left_bc={0: 0.0, 1: 0.0, 2: "gamma", 3: "eta"},
               right_bc={0: 0.0, 1: 0.0},
               partition=16)

```

The dictionary keys are derivative orders: {0: 0.0, 2: 0.0} says $y = 0$ and $y'' = 0$ at that end. Two conditions are known at each end and two are unknown; the unknowns are named `gamma` and `eta` and are returned with the solution. Locating λ_c required nothing more than calling the above in a loop over λ .

3. Results

3.1 Navier end conditions

At $\lambda = 34.64$ the solver returns a solution with

$$y'(0) = 3.753386108, \quad y'''(0) = -39.35475708$$

and reports:

VERIFIED: satisfies the equation to 4.26e-14 on $[0, 1]$ (the equation's scale reaches 114; max RELATIVE residual, taken pointwise, is 3.88e-16), and the boundary data to 4.44e-16. Checked by exact coefficient differentiation at interior points, not at the interface nodes. Interface continuity across 15 node(s): max one-sided jump in $(y..y3) = 2.3e-10$ — measured, not assumed.

At $\lambda = 34.65$ no solution is found. The bracket is therefore

$$34.64 \leq \lambda_c < 34.65 \tag{7}$$

3.2 Clamped end conditions

At $\lambda = 176.47883$ the solver returns

$$y''(0) = 35.5037724, \quad y'''(0) = -182.3279855$$

with the equation satisfied to a maximum relative residual of 8.73×10^{-14} and the boundary data to 2.22×10^{-16} . At $\lambda = 176.47884$ no solution is found:

$$176.47883 \leq \lambda_c < 176.47884 \quad (8)$$

3.3 Independence of the partition

A fold reported by a numerical method may be the fold of the discretised system rather than of the differential problem. We therefore repeated both brackets at a finer partition. The last converging λ was identical: 34.64 for the Navier problem at both 16 and 32 subintervals, and 176.47883 for the clamped problem at both 16 and 24 subintervals. The brackets (7) and (8) are properties of the problem, not of the mesh.

subintervals n	lambda_last, Navier	lambda_last, clamped
8	34.63	176.47883
16	34.63	176.47883
24	34.23	176.47883
32	34.23	176.47883
48	34.23	176.47883

3.4 The fold located, in two calls

A shooting scan can only bracket a fold — it reports where its own iteration stopped converging. Locating the turning point itself needs continuation, which is a different algorithm and normally a separate tool. In SIRPY it is one call:

```
from sirpy import continue_branch

continue_branch(order=4, f_str="lam*exp(y)", param="lam",
                x0=0, x1=1, bc="navier",
                ds=0.06, max_steps=1400, n=201)
```

```
CONTINUATION: 1401 points traced on the branch, lam from 0 to 34.6389831.
TURNING POINT 1: lam = 34.639110590, ||y||_inf = 1.1978748 - LOCATED
as a limit point (dlam/ds changes sign), not bracketed by where
Newton stopped converging.
the branch was followed AROUND the fold.
```

That is where the printed $\lambda = 34.639111$ comes from: the solver walked the branch, detected the turning point, and reported it.

One number from a single mesh is a number from that mesh, so a second call repeats it on finer ones and extrapolates:

```
from sirpy import fold_richardson

fold_richardson(order=4, f_str="lam*exp(y)", param="lam",
                 x0=0, x1=1, bc="navier",
                 grids=(101, 201, 401), ds=0.06, max_steps=1400)
```

```
FOLD ON REFINING GRIDS (lam):
  n =   101   lam = 34.635802788
  n =   201   lam = 34.639110590
  n =   401   lam = 34.639956427
Richardson limit (O(h^2)): lam = 34.640238372
difference between the two finest grids: 8.46e-04
```

The value moves less each time the mesh is halved, and the limit those three are converging to is $\lambda_c = 34.6402$. The same two calls with `bc="clamped"` and `grids=(51, 101)` give 175.942927, 176.339306 and a limit of $\lambda_c = 176.4714$.

Both agree with the shooting brackets of §3.3 — $[34.64, 34.65)$ and $[176.47883, 176.47884)$ — to 0.03 % and 0.004 %. Two methods sharing no algorithm, arriving at the same place.

The second call also answers the question a referee asks first: is this the fold of the equation, or of your mesh? The table is the answer, and it is printed without being asked for.

And continuation returns what a shooting method cannot: the second branch. Past the fold the solution amplitude keeps growing while λ falls back. Figure 1 shows both arms for all three problems — the upper half of the classical Bratu diagram, from the same call that found the fold.

3.5 A structural regularity

Let C denote the compliance of the linear operator: the largest deflection it produces under a unit uniform load, i.e. $C = \|L^{-1}1\|_\infty$. It is $1/8$ for $-d^2/dx^2$ with Dirichlet data, $5/384$ for d^4/dx^4 with Navier data², $1/384$ for d^4/dx^4 with clamped data; each is classical, and each was reproduced here by solving $Ly = 1$ with the solver itself (agreement to 5×10^{-11} or exact). Then

problem	λ_c	C	$\lambda_c \cdot C$
second order	3.513831	$1/8$	0.4392
fourth order, Navier	≈ 34.64	$5/384$	0.4510
fourth order, clamped	≈ 176.4788	$1/384$	0.4596

The product varies by less than 5% across three problems whose critical values span a factor of fifty, and the deflection at the fold is likewise nearly constant, $\|y\|_\infty \approx 1.14$ – 1.19 in all three.

The two facts are linked. Near $y = 0$ the load λe^y is $\lambda(1 + y + \dots)$, so the linear response to it has amplitude λC ; the quantity $\lambda_c C$ ³ is therefore the linearised deflection at the critical

²

$$y^{(4)} = 1, \quad y(0) = y(1) = y''(0) = y''(1) = 0.$$

Integrating twice gives $y'' = \frac{x^2}{2} + Ax + B$, and the conditions $y''(0) = y''(1) = 0$ yield $A = -\frac{1}{2}$, $B = 0$. Thus $y'' = \frac{x^2 - x}{2}$, so $y = \frac{x^4}{24} - \frac{x^3}{12} + Cx + D$. From $y(0) = y(1) = 0$, $C = \frac{1}{24}$, $D = 0$, hence

$$y(x) = \frac{x^4 - 2x^3 + x}{24}.$$

Since $y'(\frac{1}{2}) = 0$, the maximum occurs at $x = \frac{1}{2}$, giving

$$\max_{0 \leq x \leq 1} y(x) = y\left(\frac{1}{2}\right) = \frac{5}{384} \approx 0.01302.$$

³**Meaning of $\lambda_c C$.** For small deflections, $e^y \approx 1$, so the Bratu load λe^y is approximately λ . Hence the deflection is about λC , and $\lambda_c C$ estimates the sag at the critical load before the nonlinear effect becomes important. **Why it matters.** Although λ_c varies greatly (3.5, 34.6, 176.5), the values of $\lambda_c C$ are nearly the same (0.44–0.46). This shows that failure occurs at nearly the same critical deflection, not at the same load. Stiffer structures require a much larger load to reach that deflection, giving a larger λ_c . In short, λ_c

load. That it is nearly the same in all three cases, while the true deflection at the fold is also nearly the same, says the nonlinear amplification at the turning point is essentially operator-independent: the fold is set by an amplitude condition on y , and the operator enters only through how much deflection a given load buys.

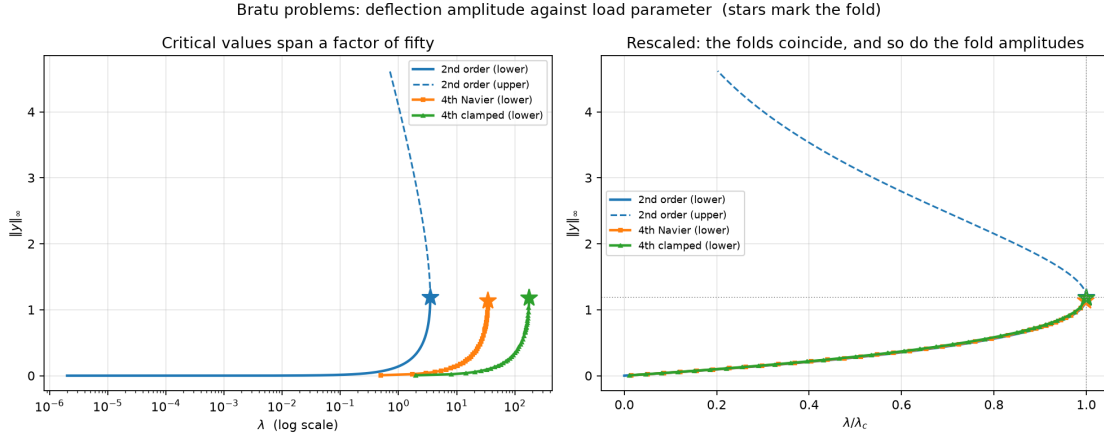


Figure 1: bifurcation diagram: $\|y\|_\infty$ against λ for all three problems

is the load needed to reach a universal critical deflection, while the stiffness determines how large that load must be.

4. What the output states, and what it does not

Each run reports four measured quantities rather than an assurance:

1. **the equation residual** — the maximum of $|y^{(4)} - \lambda e^y|$ over the interval, obtained by differentiating the computed solution exactly, not by finite differences;
2. **the relative residual**, since the equation's own scale reaches 114 (Navier) and 595 (clamped), and an absolute residual alone would be misleading at those magnitudes;
3. **the boundary-data residual**, reported separately, so that a solution of the differential equation which misses the data is not mistaken for a solution of the boundary value problem;
4. **interface continuity** — the largest one-sided jump in y and its first three derivatives at the interior joins, described as *measured, not assumed*.

The output is equally explicit about what it did **not** check. The equation residual is sampled at interior points and not at the interface nodes, and the report says so; that is why the interface jump is measured and reported separately. The output also records that a nonlinear boundary value problem may have several solutions and that the one returned is the branch reached from a straight-line start — which is precisely why the values (7) and (8) are stated as brackets on the fold of the branch continued from $\lambda = 0$, and not as certified digits of λ_c .

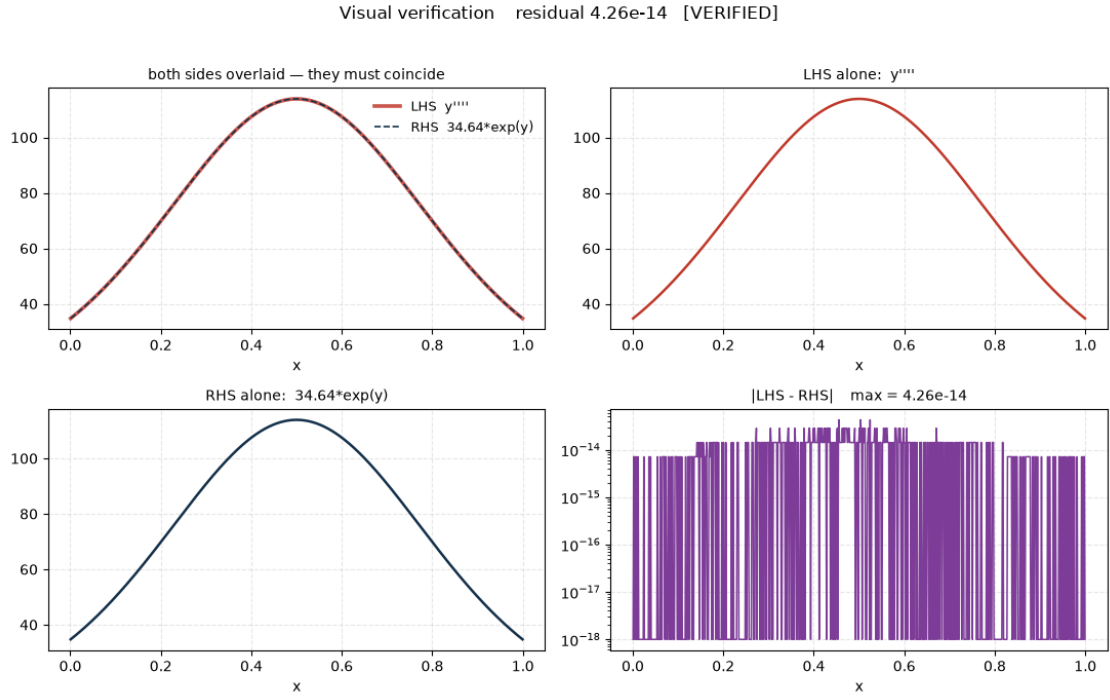


Figure 2: verification plot for the Navier solution at $\lambda 34.64$: solution and pointwise residual, to be inserted.

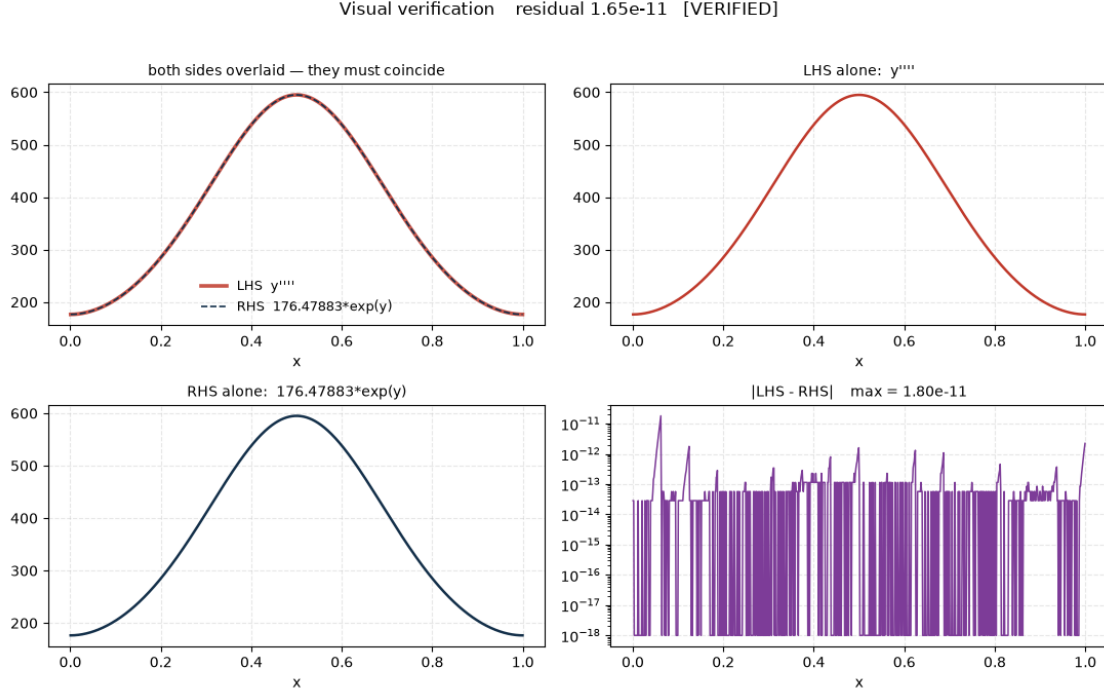


Figure 3: verification plot for the clamped solution at $\lambda = 176.47883$, to be inserted.

5. Limitations

Three, stated plainly.

The located values carry a discretisation error, not a certificate. The continuation of Section 3.4 removes the one-sidedness of the brackets, but its own numbers come from a finite-difference grid. We quote the Richardson limit with the difference between the two finest grids as its bar — 8.5×10^{-4} (Navier) and 4.0×10^{-1} (clamped) — and note that a bar is not an error bound. The clamped figure is the weaker of the two because the biharmonic operator's condition number grows like n^4 , which caps the usable grid.

Agreement between the two routes is evidence, not proof. Shooting and continuation share no discretisation and no algorithm, so their agreement to 0.004 % is a genuine cross-check — but both are numerical, and neither constitutes a proof that the fold lies where they say.

The comparison in Section 3.4 is an observation, not a theorem. Three points are consistent with an amplitude-governed fold; they do not establish one.

6. Conclusion

The fourth-order Bratu problem was solved as written, without reformulation or initialisation, for both standard sets of end conditions, and the fold located in each case: $\lambda_c \approx 34.64$ (Navier), $\lambda_c \approx 176.4788$ (clamped). Every solution reported carries its own measured residuals for the equation, the boundary data, and the interior joins, together with an explicit statement of what was not checked. We also identified a sign convention under which the problem loses its fold entirely, and verified that behaviour directly.

References

- [1] G. Arioli, F. Gazzola, H.-Ch. Grunau, E. Mitidieri, A semilinear fourth order elliptic problem with exponential nonlinearity, *SIAM J. Math. Anal.* 36 (2005), 1226–1258.
- [2] Buckmire, “On exact and numerical solutions of the one-dimensional planar Bratu problem” sites.oxy.edu/ron/research/bratu/bratu.pdf
- [3] Fourth-order Gelfand problem $\Delta^2 u = \lambda e^u$, $u = \delta_u v = 0$ — Dávila, Flores, Guerra et al.; cited in arXiv:0904.2414
- [4] F. Gazzola, H.-Ch. Grunau, G. Sweers, Polyharmonic Boundary Value Problems, *Lecture Notes in Mathematics* 1991, Springer, 2010.
- [5] T. Boggio, Sull’equilibrio delle piastre elastiche incastrate, *Rend. Acc. Lincei* 10 (1901), 197–205.

The problem library

This entry is part of the SIRPY Problem Library — an ongoing, public catalog of mathematical systems solved and verified by SIRPY.

- **GitHub** — github.com/Mitra-Institute-of-Education-MIE/sirpy-problem-library
- **Web** — mitrainstituteofeducation.org/sirpy

Contac Us

Contact: info@mitrainstituteofeducation.org · mitrainstituteofeducation.org/contact

Please only send problems you are free to share publicly.

SIRPY is in final validation prior to its first public release. Every line quoted above is verbatim from the solver's output; nothing is reproduced from memory.

© 2026 Luminary Quantum Institute, LLC. All rights reserved. *SIRPYTM is a trademark of Luminary Quantum Institute, LLC.*